

```
import pandas as pd
```

```
Start coding or generate with AI.
```

```
df = pd.read_csv("train (1).csv")
```

```
df.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley
0	1	60	RL	65.0	8450	Pave	Na
1	2	20	RL	80.0	9600	Pave	Na
2	3	60	RL	68.0	11250	Pave	Na
3	4	70	RL	60.0	9550	Pave	Na
4	5	60	RL	84.0	14260	Pave	Na

5 rows x 81 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     1460 non-null   int64
1   MSSubClass             1460 non-null   int64
2   MSZoning               1460 non-null   object
3   LotFrontage            1201 non-null   float64
4   LotArea                1460 non-null   int64
5   Street                 1460 non-null   object
6   Alley                  91 non-null     object
7   LotShape               1460 non-null   object
8   LandContour            1460 non-null   object
9   Utilities              1460 non-null   object
10  LotConfig              1460 non-null   object
11  LandSlope              1460 non-null   object
12  Neighborhood            1460 non-null   object
13  Condition1             1460 non-null   object
14  Condition2             1460 non-null   object
```

15	BldgType	1460	non-null	object
16	HouseStyle	1460	non-null	object
17	OverallQual	1460	non-null	int64
18	OverallCond	1460	non-null	int64
19	YearBuilt	1460	non-null	int64
20	YearRemodAdd	1460	non-null	int64
21	RoofStyle	1460	non-null	object
22	RoofMatl	1460	non-null	object
23	Exterior1st	1460	non-null	object
24	Exterior2nd	1460	non-null	object
25	MasVnrType	588	non-null	object
26	MasVnrArea	1452	non-null	float64
27	ExterQual	1460	non-null	object
28	ExterCond	1460	non-null	object
29	Foundation	1460	non-null	object
30	BsmtQual	1423	non-null	object
31	BsmtCond	1423	non-null	object
32	BsmtExposure	1422	non-null	object
33	BsmtFinType1	1423	non-null	object
34	BsmtFinSF1	1460	non-null	int64
35	BsmtFinType2	1422	non-null	object
36	BsmtFinSF2	1460	non-null	int64
37	BsmtUnfSF	1460	non-null	int64
38	TotalBsmtSF	1460	non-null	int64
39	Heating	1460	non-null	object
40	HeatingQC	1460	non-null	object
41	CentralAir	1460	non-null	object
42	Electrical	1459	non-null	object
43	1stFlrSF	1460	non-null	int64
44	2ndFlrSF	1460	non-null	int64
45	LowQualFinSF	1460	non-null	int64
46	GrLivArea	1460	non-null	int64
47	BsmtFullBath	1460	non-null	int64
48	BsmtHalfBath	1460	non-null	int64
49	FullBath	1460	non-null	int64
50	HalfBath	1460	non-null	int64
51	BedroomAbvGr	1460	non-null	int64
52	KitchenAbvGr	1460	non-null	int64
53	KitchenQual	1460	non-null	object

```
df.describe()
```

	Id	MSSubClass	LotFrontage	LotArea	Overall
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.
mean	730.500000	56.897260	70.049958	10516.828082	6.
std	421.610009	42.300571	24.284752	9981.264932	1.
min	1.000000	20.000000	21.000000	1300.000000	1.
25%	365.750000	20.000000	59.000000	7553.500000	5.
50%	730.500000	50.000000	69.000000	9478.500000	6.
75%	1095.250000	70.000000	80.000000	11601.500000	7.
max	1460.000000	190.000000	313.000000	215245.000000	10.

8 rows x 38 columns

```
df.isnull().sum()
```

	0
Id	0
MSSubClass	0
MSZoning	0
LotFrontage	259
LotArea	0
...	...
MoSold	0
YrSold	0
SaleType	0
SaleCondition	0
SalePrice	0

81 rows × 1 columns

dtype: int64

```
df.shape
```

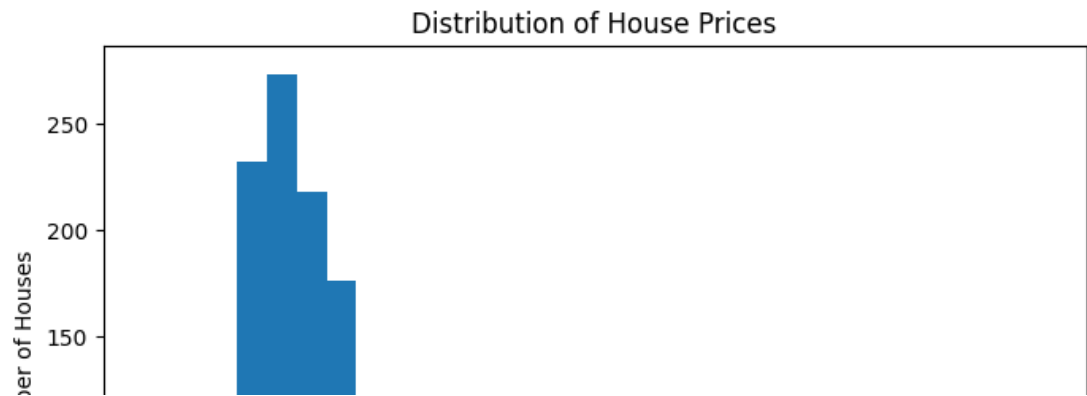
```
(1460, 81)
```

```
df.columns
```

```
Index(['Id', 'MSSubClass', 'MSZoning', 'LotFrontage',  
      'LotArea', 'Street',  
      'Alley', 'LotShape', 'LandContour', 'Utilities',  
      'LotConfig',  
      'LandSlope', 'Neighborhood', 'Condition1',  
      'Condition2', 'BldgType',  
      'HouseStyle', 'OverallQual', 'OverallCond',  
      'YearBuilt', 'YearRemodAdd',  
      'RoofStyle', 'RoofMatl', 'Exterior1st', 'Exterior2nd',  
      'MasVnrType',  
      'MasVnrArea', 'ExterQual', 'ExterCond', 'Foundation',  
      'BsmtQual',  
      'BsmtCond', 'BsmtExposure', 'BsmtFinType1',  
      'BsmtFinSF1',  
      'BsmtFinType2', 'BsmtFinSF2', 'BsmtUnfSF',  
      'TotalBsmtSF', 'Heating',  
      'HeatingQC', 'CentralAir', 'Electrical', '1stFlrSF',  
      '2ndFlrSF',  
      'LowQualFinSF', 'GrLivArea', 'BsmtFullBath',  
      'BsmtHalfBath', 'FullBath',  
      'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr',  
      'KitchenQual',  
      'TotRmsAbvGrd', 'Functional', 'Fireplaces',  
      'FireplaceQu', 'GarageType',  
      'GarageYrBlt', 'GarageFinish', 'GarageCars',  
      'GarageArea', 'GarageQual',  
      'GarageCond', 'PavedDrive', 'WoodDeckSF',  
      'OpenPorchSF',  
      'EnclosedPorch', '3SsnPorch', 'ScreenPorch',  
      'PoolArea', 'PoolQC',  
      'Fence', 'MiscFeature', 'MiscVal', 'MoSold', 'YrSold',  
      'SaleType',  
      'SaleCondition', 'SalePrice'],  
      dtype='object')
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,5))
plt.hist(df["SalePrice"], bins=30)
plt.title("Distribution of House Prices")
plt.xlabel("Sale Price")
plt.ylabel("Number of Houses")
plt.show()
```

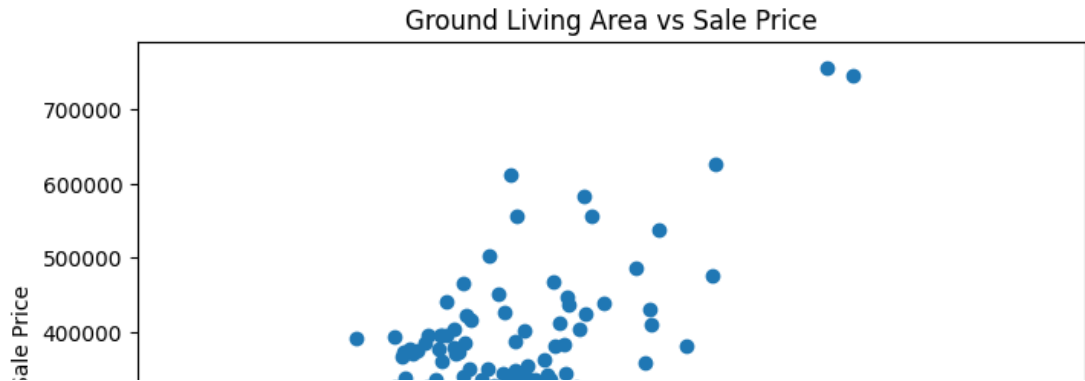


```
df.corr(numeric_only=True) ["SalePrice"].sort_values(ascending=False)
```

	SalePrice
SalePrice	1.000000
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.500000

YearBuilt	0.522897
YearRemodAdd	0.507101
GarageYrBlt	0.486362
MasVnrArea	0.477493
Fireplaces	0.466929
BsmtFinSF1	0.386420
LotFrontage	0.351799
WoodDeckSF	0.324413
2ndFlrSF	0.319334
OpenPorchSF	0.315856
HalfBath	0.284108
LotArea	0.263843
BsmtFullBath	0.227122
BsmtUnfSF	0.214479
BedroomAbvGr	0.168213
ScreenPorch	0.111447
PoolArea	0.092404
MoSold	0.046432
3SsnPorch	0.044584
BsmtFinSF2	-0.011378
BsmtHalfBath	-0.016844
MiscVal	-0.021190
Id	-0.021917
LowQualFinSF	-0.025606
YrSold	-0.028923
OverallCond	-0.077856
MSSubClass	-0.084284
EnclosedPorch	-0.128578
KitchenAbvGr	-0.135907

```
plt.figure(figsize=(8,5))
plt.scatter(df["GrLivArea"],df["SalePrice"])
plt.xlabel("Ground Living Area")
plt.ylabel("Sale Price")
plt.title("Ground Living Area vs Sale Price")
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

```
X = df[["GrLivArea"]]
Y = df["SalePrice"]
```

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=42
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, Y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```
Y_pred = model.predict(X_test)
```

```
print(r2_score(Y_test, Y_pred))
```

```
0.5542632452871117
```

```
print(Y_pred[:10])
```

```
[134357.9562804  293625.79843699 130258.39792373 195441.37579.  
190931.86160243  89467.79227487 225163.17388062 174123.67234  
89467.79227487 167666.86792833]
```

```
comparison = pd.DataFrame({
    "Actual Price": Y_test.values,
    "Predicted Price": Y_pred
})

comparison.head(10)
```

	Actual Price	Predicted Price
0	154500	134357.956280
1	325000	293625.798437
2	115000	130258.397924
3	159000	195441.375795
4	315500	190931.861602
5	75500	89467.792275
6	311500	225163.173881
7	146000	174123.672340
8	84500	89467.792275
9	135500	167666.867928

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(Y_test, Y_pred)
mse = mean_squared_error(Y_test, Y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(Y_test, Y_pred)

print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R2 Score:", r2)
```

```
Mean Absolute Error: 38341.204476421066
Mean Squared Error: 3418946311.180807
Root Mean Squared Error: 58471.75652552955
R2 Score: 0.5542632452871117
```

```
import matplotlib.pyplot as plt

plt.scatter(Y_test, Y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted House Prices")
plt.show()
```

